🧪 TP SQL – Gestion Bancaire (GROUP BY, HAVING, INNER JOIN)

📘 Contexte

Une banque souhaite analyser les **comptes de ses clients et les opérations bancaires** réalisées.
La base de données contient des informations sur les **clients**, leurs **comptes** et les **opérations** effectuées.

1️⃣ Création de la base de données

```
CREATE DATABASE banque;
USE banque;
```

2️⃣ Création des tables

Table clients

```
CREATE TABLE clients (
   id_client INT AUTO_INCREMENT PRIMARY KEY,
   nom VARCHAR(50),
   prenom VARCHAR(50),
   ville VARCHAR(50)
);
```

Table comptes

```
CREATE TABLE comptes (
   id_compte INT AUTO_INCREMENT PRIMARY KEY,
   id_client INT,
   type_compte VARCHAR(50),
   solde DECIMAL(10,2),
   FOREIGN KEY (id_client) REFERENCES clients(id_client)
);
```

Table operations

```
CREATE TABLE operations (
   id_operation INT AUTO_INCREMENT PRIMARY KEY,
   id_compte INT,
   type_operation VARCHAR(50),
   montant DECIMAL(10,2),
   date_operation DATE,
   FOREIGN KEY (id_compte) REFERENCES comptes(id_compte)
);
```

3️⃣ Insertion des données

Clients

```
INSERT INTO clients (nom, prenom, ville) VALUES
('Mensah','Koffi','Lomé'),
('Adjovi','Afi','Cotonou'),
('Tchala','Yaw','Kara'),
('Amadou','Fatou','Lomé');
```

Comptes

```
INSERT INTO comptes (id_client, type_compte, solde) VALUES
(1,'Epargne',150000),
(2,'Courant',80000),
(3,'Epargne',200000),
(4,'Courant',50000);
```

Opérations

```
INSERT INTO operations (id_compte, type_operation, montant, date_operation) VALUES
(1,'Depot',50000,'2025-02-01'),
(1,'Retrait',20000,'2025-02-03'),
(2,'Depot',30000,'2025-02-02'),
(3,'Depot',100000,'2025-02-05'),
(3,'Retrait',40000,'2025-02-06'),
(4,'Depot',20000,'2025-02-07');
```

4️⃣ Travaux à faire

Exercice 1 : Liste des clients et leurs comptes (Jointure interne)

```
SELECT clients.nom, clients.prenom, comptes.type_compte, comptes.solde
FROM clients
INNER JOIN comptes
ON clients.id_client = comptes.id_client;
```

Exercice 2 : Liste des opérations avec le nom du client

```
SELECT clients.nom, comptes.type_compte, operations.type_operation, operations.montant
FROM operations
INNER JOIN comptes ON operations.id_compte = comptes.id_compte
INNER JOIN clients ON comptes.id_client = clients.id_client;
```

Exercice 3 : Montant total des opérations par compte

```
SELECT id_compte, SUM(montant) AS total_operations
FROM operations
GROUP BY id_compte;
```

Exercice 4 : Montant total déposé par compte

```
SELECT id_compte, SUM(montant) AS total_depot
FROM operations
WHERE type_operation = 'Depot'
GROUP BY id_compte;
```

Exercice 5 : Nombre d'opérations par compte

```
SELECT id_compte, COUNT(*) AS nombre_operations
FROM operations
GROUP BY id_compte;
```

Exercice 6 : Comptes ayant effectué plus de 2 opérations (HAVING)

```
SELECT id_compte, COUNT(*) AS nombre_operations
FROM operations
GROUP BY id_compte
HAVING COUNT(*) > 2;
```

Exercice 7 : Clients ayant déposé plus de 50000

```
SELECT clients.nom, SUM(operations.montant) AS total_depot
FROM operations
INNER JOIN comptes ON operations.id_compte = comptes.id_compte
INNER JOIN clients ON comptes.id_client = clients.id_client
WHERE operations.type_operation = 'Depot'
GROUP BY clients.nom
HAVING SUM(operations.montant) > 50000;
```

Exercice 8 : Solde moyen des comptes par type

```
SELECT type_compte, AVG(solde) AS solde_moyen
FROM comptes
GROUP BY type_compte;
```

🎯 Objectifs pédagogiques

À la fin de ce TP, l'étudiant doit savoir :

- créer une **base de données bancaire**
- utiliser les **fonctions d'agrégation (SUM, AVG, COUNT)**
- utiliser **GROUP BY**
- filtrer les groupes avec **HAVING**
- utiliser **INNER JOIN**
- analyser des **données bancaires**